

Для начала работы мне было необходимо собрать фотографии. Мной были собраны фотографии телефона. Работа производилась над двумя фотографиями представленными ниже.



В первую очередь было необходимо загрузить изображение в оттенках серого.
$$g(x, y) = 0.2989 * r(x, y) + 0.5870 * g(x, y) + 0.1140 * b(x, y)$$

Для того, чтобы обеспечить лучшую устойчивость к шуму необходимо было предварительно обработать изображение осуществив нормализацию яркости при помощи следующей функции.

$$g(x, y) = \min(255, \max(0, I(x, y) - \mu + 128))$$

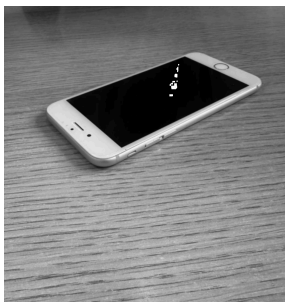
После нормализации, было создано ядро Гауса при помощи функции:

$$G(x, y) = (1 / (2 * \pi * \sigma^2)) * \exp(-(x^2 + y^2) / (2 * \sigma^2))$$

и написана функция для свертки изображения с ядром:

$$O(i, j) = \sum_{k=-h/2, k=h/2} \sum_{l=-w/2, l=w/2} I(i + k, j + l) * K(k, l)$$

После чего было применено Гауссовское размытие. Результаты работы можно увидеть ниже на изображении в оттенках серого.



Следующим этапом работы было обнаружение и описание ключевых точек. Мной использовался алгоритм Хариса для обнаружения углов. Для начала я еще раз написал функцию свертки для дальнейшего подсчета градиентов, посчитал градиенты и вычислил ответ Хариса для обнаружения углов при помощи функции:

$$R(x, y) = \det(M(x, y)) - k * (\text{trace}(M(x, y)))^2$$

$$M(x, y) = \begin{bmatrix} \sum(I_x(x, y) * I_x(x, y)) & \sum(I_x(x, y) * I_y(x, y)) \\ \sum(I_x(x, y) * I_y(x, y)) & \sum(I_y(x, y) * I_y(x, y)) \end{bmatrix}$$

После чего была реализована функция подавления нерелевантных углов в массиве значений всех углов, поскольку мне было необходимо найти только локальные максимумы, что выглядело следующим образом:

Точка (x, y) - угол, если:

- $R(x, y) > \text{threshold}$;
- $R(x, y) \geq R(x + dx, y + dy)$ для всех $(dx, dy) \in \{-1, 0, 1\}^2$ за исключением $(0, 0)$

Следующим этапом было описание дескрипторов для ключевых точек:

Дескриптор $D(kp)$ для ключевой точки $kp = (x, y)$:

- $D(kp) = [I(x + i - \text{offset}, y + j - \text{offset}) \mid i \in \{0, \dots, \text{patch_size} - 1\}, j \in \{0, \dots, \text{patch_size} - 1\}]$
- $D(kp) = D(kp) / \|D(kp)\|_1$ (нормализация с помощью L1-нормы)

После описания дескрипторов ключевые точки были сопоставлены между изображениями. Для этого было вычислено евклидово расстояние между двумя дескрипторами при помощи функции:

$$d(x, y) = \sqrt{\sum (x_i - y_i)^2}$$

После чего на его основании были сопоставлены ключевые точки. Сопоставление было произведено следующим образом:

Совпадение (i, j) :

- i - индекс ключевой точки из первого изображения.
- j - индекс ключевой точки из второго изображения, которая является наилучшим совпадением для ключевой точки i .

Наилучшее совпадение j для ключевой точки i :

- $j = \text{argmin}(d(\text{desc1}(i), \text{desc2}(j)))$
- $d(\text{desc1}, \text{desc2})$ - евклидово расстояние между дескрипторами desc1 и desc2 .

Для повышения точности был реализован собственный вариант алгоритма RANSAC.

Для его реализации было создана матрица для аффинного преобразования

$$\begin{bmatrix} x' & y' & 1 \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} * \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix}$$

Реализовано умножение точки на матрицу преобразования. Реализованы функции умножения матриц, векторного умножения, транспонирования, решения системы уравнений методом наименьших квадратов.

Говоря о фильтрации изображения, стоит сказать, что для изображений обработанных Гаусовским фильтром количество соответствий составило 132, в то время, как для изображений без фильтра их количество было равно 2814. Это говорит о том, что фильтрация изображения помогла снизить возможные ошибки, связанные с неверной

идентификацией ключевых точек. Более того, алгоритм RANSAC сработал более эффективно. Стоит сказать, что используемые методы не самые эффективные, поскольку код запускается и обрабатывает изображение довольно долго.

Теоретический анализ алгоритмов:

Аналитический обзор

1. Преобразование RGB в градации серого

Для оптимизации можно использовать векторизованные операции с помощью библиотек, таких как NumPy, для ускорения процесса.

2. Нормализация яркости

Оптимизация: Использование NumPy для параллельной обработки и суммирования значений яркости.

3. Гауссово размытие

Оптимизация:

Использование быстрой свертки (Fast Convolution) с помощью методов, таких как свертка в частотной области (FFT).

Применение оптимизированных библиотек для обработки изображений, таких как OpenCV.

4. Обнаружение градиентов

Оптимизация: Можно использовать векторизованные операции с помощью NumPy или специализированные библиотеки для обработки изображений.

5. Ответ Хариса

Оптимизация: Использование матричных операций для уменьшения количества циклов.

6. Подавление нерелевантных углов

Оптимизация: Можно использовать методы, такие как "параллельное подавление", чтобы уменьшить количество проверок.

7. Извлечение дескрипторов

Оптимизация: Использование NumPy для извлечения и нормализации дескрипторов.

8. Сопоставление ключевых точек

Оптимизация: Использование более эффективных структур данных, таких как KD-деревья или FLANN, для ускорения поиска ближайших соседей.

9. Фильтрация RANSAC

Оптимизация: Уменьшение количества итераций K, если это возможно, или использование более эффективных методов для оценки модели.

10. Применение аффинного преобразования

Оптимизация: Использование векторизованных операций для применения преобразования ко всем точкам одновременно.

Как результат, большое количество циклов и отсутствие оптимизации во всех этапах приводит к долгой обработке двух изображений еще на данном этапе работы, что говорит о том, что при использовании большого количества изображений для реконструкции 3d модели запуск кода может занимать в разы больше времени.

Возможные улучшения и перспективы развития подхода:

Использование глубокого обучения:

Применение нейронных сетей для обнаружения ключевых точек и дескрипторов, что может повысить точность и устойчивость к шуму.

Оптимизация алгоритмов:

Разработка адаптивных алгоритмов, которые могут подстраиваться под характеристики входных изображений, такие как уровень шума и контрастность.

Параллельная обработка:

Использование многопоточности или GPU для ускорения вычислений, нескольких изображений параллельно.

В моем подходе также стоило реализовать возможность настройки параметров, таких как размер ядра и порога, после чего сравнивать получаемые результаты, что в дальнейшем могло бы сказаться на качестве реализации самой 3d модели.